cinescope

Cinescope

Catalogue de films full-stack

MODULE

OPSCI - Operations et services informatiques

AUTEUR (BINOME)

Wassim Garbouj - L3 Informatique

(travail en binome conformement au sujet)

DATE

29 April 2026

TRAVAUX INTEGRES

* TME 2 - Backend FastAPI + endpoints REST
* TME 3 - Donnees externes (TMDB) + export JSON / CSV
* TME 4 - Frontend HTML/CSS/JS + integration API
* TME 5 - Conteneurisation Docker (multi-services)
* TME 7 - Orchestration Kubernetes + autoscaling
* Projet final - PostgreSQL + Ingress + CI/CD GitLab

Universite - 2025/2026

Rapport de projet

00

Sommaire

01 Introduction et contexte
02 Choix techniques et justifications
03 Architecture generale
04 Backend - FastAPI
05 Base de donnees - PostgreSQL
06 Frontend - HTML/CSS/JS
07 Conteneurisation - Docker
08 Orchestration - Kubernetes
09 CI/CD - GitLab
10 Tests
11 Difficultes rencontrees et solutions
12 Bilan et perspectives
A Annexes - commandes utiles

01

Introduction et contexte

Ce rapport presente **Cinescope**, une application web de catalogue de films developpee dans le cadre du module OPSCI. Le projet a ete construit progressivement au fil des seances de TME (TME 2 a TME 7) puis assemble en un projet final complet, conformement a l'esprit du sujet qui demande de rediger le rapport au fur et a mesure.

Le travail est realise **en binome** comme le demande explicitement le sujet, afin de se rapprocher d'un contexte reel de developpement. L'objectif n'est pas de produire l'application la plus complexe possible mais de demontrer notre capacite a concevoir, justifier, implementer et deployer une application complete dans une demarche proche des pratiques professionnelles.

Fonctionnalites principales

- Catalogue de films avec affichage en grille
- Barre de recherche multi-champs (titre, realisateur, genre, description)
- Page de detail par film (cast, trailer YouTube, note, duree)
- Export des donnees en JSON et CSV
- Endpoint **/status** pour observer l'etat de l'application
- Mode mock automatique si la cle TMDB n'est pas fournie

Perimetre technique couvert

- Backend Python avec FastAPI (programmation asynchrone)
- Base de donnees PostgreSQL 16 (table films, requetes parametrees)
- Frontend HTML/CSS/JavaScript natif (pas de framework)
- Conteneurisation Docker des trois services (db, back, front)
- Orchestration Kubernetes (Pods, Services, Ingress, ConfigMap, Secret, HPA)
- Pipeline CI/CD GitLab (test, build, deploy) avec Postgres en service

02

Choix techniques et justifications

Le sujet laisse les technologies au libre choix tant qu'elles sont coherentes et justifiees. Voici nos choix et les raisons qui nous ont amenes a les retenir.

Composant	Choix et justification
Backend	FastAPI (Python 3.11) - asynchrone natif, OpenAPI auto-genere, syntaxe simple a prendre en main
Base de donnees	PostgreSQL 16 - SQL standard largement utilise en entreprise, persistance via volume, types strict
Frontend	HTML / CSS / JavaScript vanilla - pas de chaine de build, focus sur la comprehension du DOM et
Donnees	API externe TMDB (option A du sujet) - donnees riches, mode mock pour developper sans cle
Conteneurisation	Docker + Docker Compose - 3 services orchestres simplement
Orchestration	Kubernetes via Minikube - pods, services, ingress, configmap, secret, HPA
CI/CD	GitLab CI (pipeline 3 stages : test, build, deploy)
Tests	pytest avec TestClient FastAPI

Pourquoi FastAPI plutot que Flask ou Django ?

FastAPI offre la programmation asynchrone (async/await) qui nous est utile pour les appels HTTP vers TMDB sans bloquer le serveur. Il genere aussi automatiquement une documentation Swagger interactive sur **/docs**, ce qui est tres pratique pour tester l'API pendant le developpement. Compare a Django, FastAPI est beaucoup plus leger et plus adapte a une API pure.

Pourquoi PostgreSQL plutot que MySQL ou MongoDB ?

Nos donnees (films) sont structurees et relationnelles, donc une base SQL est plus pertinente qu'une base NoSQL. Entre PostgreSQL et MySQL, nous avons choisi PostgreSQL car il est plus rigoureux sur les types et il supporte la clause **ON CONFLICT DO UPDATE** qui nous sert a faire un upsert propre lors du rafraichissement des donnees TMDB. C'est aussi la base la plus couramment utilisee en entreprise aujourd'hui.

Pourquoi pas de framework frontend ?

Nous avons commence le projet en HTML/CSS/JS natif au TME 4 et avons decide de ne pas ajouter de framework par la suite. Le perimetre est petit (un catalogue + une recherche + un modal), un framework comme React ou Vue aurait alourdi la chaine de build sans benefice reel. Cela nous a aussi force a comprendre **fetch()**, le DOM et **IntersectionObserver** a la main, ce qui est formateur.

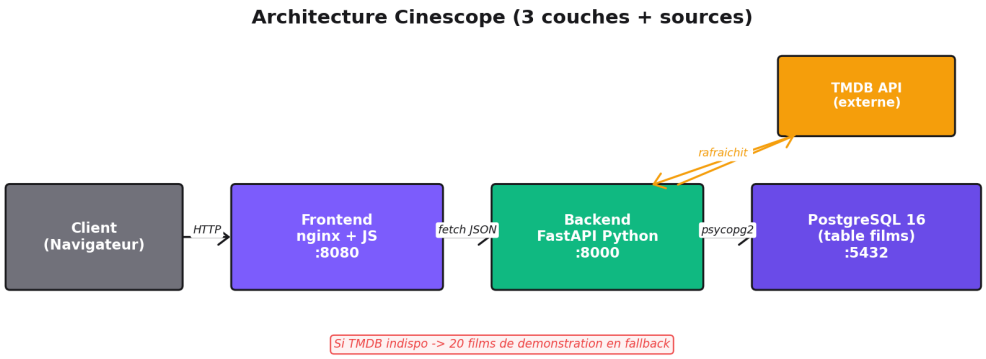
Pourquoi Kubernetes alors que Compose suffit ?

Pour une application qui tourne en local, Docker Compose est largement suffisant. Nous avons quand meme deploye sur Kubernetes parce que c'etait demande par le sujet (TME 7 et section 8 du sujet final) et parce que cela nous a permis de manipuler des concepts pros : autoscaling (HPA), separation ConfigMap / Secret, Ingress avec routage, probes de sante.

03

Architecture generale

L'application suit une architecture **3 couches** classique : un client navigateur, un frontend statique servi par nginx, un backend API FastAPI et une base de donnees PostgreSQL. TMDB est une source de donnees externe que le backend interroge en arriere-plan.



Schema general - les fleches indiquent le sens des appels

Flux de donnees

- Au demarrage, le backend cree la table **films** dans Postgres si elle n'existe pas
- Si la table est vide, il appelle TMDB pour recuperer 20 films populaires
- Si TMDB n'est pas disponible (pas de cle), il insere 20 films de demonstration
- Toutes les heures, une tache de fond rafraichit les films depuis TMDB
- Le frontend appelle **/movies**, **/search**, **/movie/{id}** via fetch()
- Les details d'un film (cast, trailer) sont mis en cache en BDD apres le premier appel

Schema de communication

```
Navigateur --HTTP--> Frontend (nginx)
Frontend --HTTP--> Backend (FastAPI)
Backend --SQL --> PostgreSQL
Backend --httpx-> TMDB API (rafraichissement uniquement)
```

04

Backend - FastAPI

Le backend tient dans un seul fichier main.py. Cette structure simple etait suffisante pour notre perimetre. Sur un projet plus gros nous aurions decoupe en plusieurs modules (routes, schemas, services).

Endpoints exposes

Route	Role
GET /hello	Healthcheck - retourne le mode (tmdb/mock) et le nombre de films en BDD
GET /status	Etat detaille - uptime, mode, nb films, intervalle de refresh
GET /movies?limit=N	Liste paginable des films (1 <= N <= 100)
GET /movie/{id}	Detail complet (realisateur, casting, trailer, synopsis)
GET /search?q=...	Recherche - tape sur TMDB si cle, sinon LIKE en BDD
GET /export/movies.json	Export JSON telechargeable
GET /export/movies.csv	Export CSV telechargeable

Programmation asynchrone

Toutes les routes sont declarees **async** et les appels HTTP vers TMDB utilisent **httpx.AsyncClient**. Cela permet au serveur de traiter d'autres requetes pendant que TMDB repond. C'etait l'un des interets pedagogiques principaux de FastAPI.

```
@app.get("/movies")
async def get_movies(limit: int = Query(default=20, ge=1, le=100)):
return get_films(limit)

async def tmdb_get(endpoint, params=None):
async with httpx.AsyncClient(timeout=10) as client:
res = await client.get(f"{TMDB_BASE}{endpoint}", ...)
return res.json()
```

Tache de rafraichissement en arriere-plan

Au demarrage de l'application (lifespan), on lance une tache asyncio qui appelle refresh_from_source() toutes les heures. Si TMDB echoue on bascule sur les films mock pour que l'application continue de fonctionner.

```
@asynccontextmanager
async def lifespan(app):
wait_for_db()
init_db()
if count_films() == 0:
```

```
    await refresh_from_source()
    task = asyncio.create_task(refresh_loop())
    yield
    task.cancel()
```

05

Base de donnees - PostgreSQL

La base contient une seule table **films** avec les colonnes essentielles pour l'affichage et la recherche, plus des champs optionnels pour le detail (cast, trailer, etc.) qui sont peuples lors du premier acces a la fiche d'un film.

Schema de la table

```
CREATE TABLE IF NOT EXISTS films (
id INTEGER PRIMARY KEY,
title TEXT NOT NULL,
year TEXT,
director TEXT,
description TEXT,
image_url TEXT,
genre TEXT,
backdrop_url TEXT,
tagline TEXT,
runtime INTEGER,
vote_average REAL,
vote_count INTEGER,
trailer_key TEXT,
cast_json TEXT, -- liste JSON serialisee
genres_json TEXT, -- idem
has_detail INTEGER DEFAULT 0,
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Upsert avec ON CONFLICT

Pour synchroniser TMDB avec notre BDD, on utilise un **upsert** (INSERT ... ON CONFLICT DO UPDATE) qui ecrit le film s'il n'existe pas et le met a jour sinon. Petite subtilite : si TMDB renvoie 'N/A' pour le realisateur (cas de l'endpoint /popular), on garde l'ancienne valeur si on l'a deja recuperee via le detail.

```
INSERT INTO films (id, title, year, director, ...)
VALUES (%(id)s, %(title)s, ...)
ON CONFLICT (id) DO UPDATE SET
title = EXCLUDED.title,
director = CASE
WHEN EXCLUDED.director <> 'N/A' THEN EXCLUDED.director
ELSE films.director
END,
updated_at = CURRENT_TIMESTAMP
```

Recherche multi-champs

Quand TMDB n'est pas joignable, la route **/search** effectue une recherche SQL avec **LIKE** sur 4 colonnes (titre, realisateur, genre, description). Les caracteres % et _ sont echappes pour eviter les wildcards involontaires.

06

Frontend - HTML/CSS/JS

Le frontend est compose de trois fichiers : index.html (la structure), style.css (theme sombre, glassmorphism, responsive) et script.js (logique fetch, recherche debounced, modal de detail). Servi par nginx sur le port 80 du conteneur (8080 en local).

Fonctionnalites principales

- Affichage en grille avec animation d'apparition (IntersectionObserver)
- Recherche avec debounce de 300ms pour eviter de spam le backend
- Modal cliquable affichant le detail (poster, casting, trailer YouTube embarque)
- Boutons pour changer la limite (6 / 12 / 20 films)
- Indicateur visuel du mode (live TMDB / mock)
- Etat 'aucun resultat' et 'erreur de connexion' avec bouton retry

Cycle d'un appel API

```
async function fetchMovies() {
showSkeletons(currentLimit); // placeholders pendant le chargement
try {
const res = await fetch(`${API_BASE}/movies?limit=${currentLimit}`);
const movies = await res.json();
movies.forEach((m, i) => grid.appendChild(createCard(m, i)));
} catch (e) {
showError("Le backend ne repond pas.");
}
}
```

Captures d'ecran

image manquante : screen_app_home.png

Page d'accueil - grille de films

image manquante : screen_app_modal.png

Modal de detail (cast + trailer YouTube)

07

Conteneurisation - Docker

Les trois composants (PostgreSQL, backend, frontend) sont encapsules dans des conteneurs Docker et orchestres avec Docker Compose. Le lancement complet du projet se fait avec une seule commande : **./start.sh** (qui appelle docker compose up --build).

Dockerfile backend

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

docker-compose.yml (extrait)

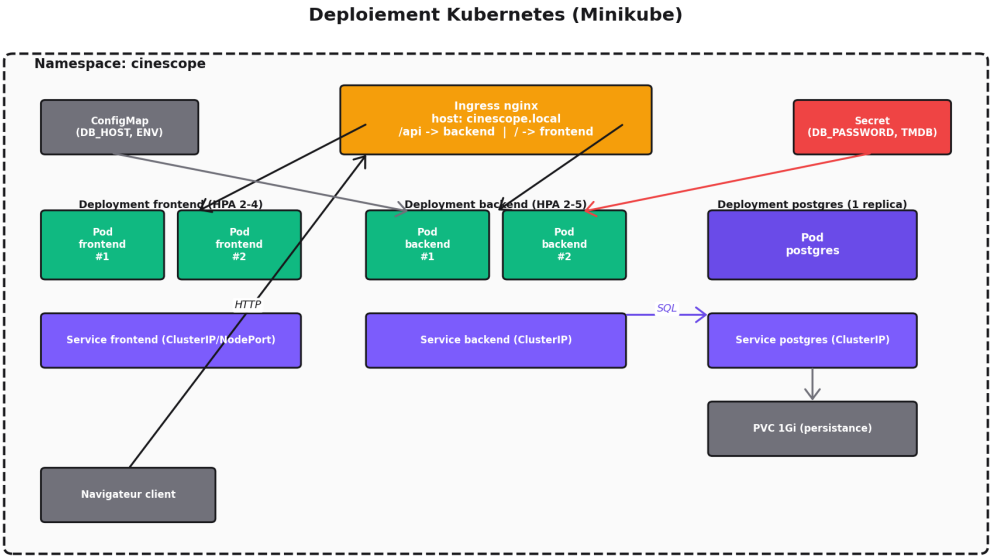
```
services:
db:
image: postgres:16-alpine
environment:
POSTGRES_DB: cinescope
POSTGRES_USER: cinescope
POSTGRES_PASSWORD: cinescope
healthcheck:
test: ["CMD-SHELL", "pg_isready -U cinescope"]
backend:
build: ./backend
depends_on:
db:
condition: service_healthy
environment:
- DB_HOST=db
- TMDB_API_KEY=${TMDB_API_KEY:-}
frontend:
build: ./frontend
ports:
- "8080:80"
```

> Le **healthcheck** sur Postgres + **depends_on: condition: service_healthy** evite que le backend demarre avant que la BDD soit prete (probleme courant avec Compose).

08

Orchestration - Kubernetes

Le projet est deployable sur un cluster Kubernetes via Minikube. Le script k8s/deploy.sh automatise la creation du namespace, l'application des manifests et l'attente que les pods soient prets.



Vue logique du deploiement K8s

Ressources deployees

Ressource	Role
Namespace	Isolation logique dans cinescope
Deployment postgres	1 replica + PVC 1Gi pour la persistance
Deployment backend	2 replicas, image films-backend:latest
Deployment frontend	2 replicas, image films-frontend:latest
Services ClusterIP	Communication interne (back -> db, front -> back)
Service frontend NodePort	Acces externe en fallback (port 30080)
Ingress nginx	Point d'entree unique : /api -> backend, / -> frontend
ConfigMap	Variables non sensibles (DB_HOST, DB_PORT, ENV, ...)
Secret	Variables sensibles (DB_PASSWORD, TMDB_API_KEY)
HPA backend	Autoscale 2-5 replicas selon CPU (cible 70%)

HPA frontend	Autoscale 2-4 replicas selon CPU (cible 70%)
Probes	readiness + liveness sur /hello (back), pg_isready (db)

Ingress - point d'entree unique

L'Ingress remplace le NodePort comme entree principale : un seul hostname (cinescope.local) sert a la fois le frontend et l'API. Les requetes vers **/api** sont redirigees vers le backend, le reste vers le frontend. Cela imite ce qu'on ferait en production avec un domaine reel et HTTPS.

```
apiVersion: networking.k8s.io/v1
kind: Ingress
spec:
ingressClassName: nginx
rules:
- host: cinescope.local
http:
paths:
- path: /api(/|$)(.*)
backend: { service: { name: backend, port: { number: 8000 } } }
- path: /
backend: { service: { name: frontend, port: { number: 80 } } }
```

Separation ConfigMap / Secret

On a separe les variables non sensibles (ConfigMap : DB_HOST, DB_NAME, ENV) des variables sensibles (Secret : DB_PASSWORD, TMDB_API_KEY). C'est une bonne pratique : les Secrets peuvent etre audites et chiffres separement, et seuls les pods qui en ont besoin y ont acces via RBAC.

Autoscaling

Le HPA (HorizontalPodAutoscaler) ajuste automatiquement le nombre de replicas du backend (2 a 5) et du frontend (2 a 4) selon l'utilisation CPU moyenne. Necessite l'addon metrics-server, active automatiquement par notre script de deploiement.

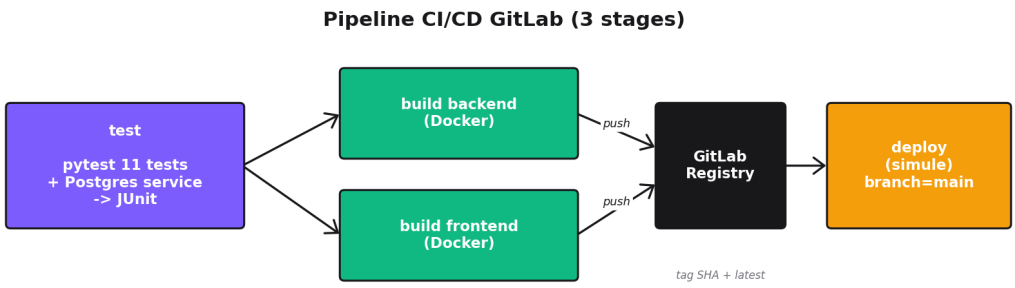
image manquante : screen_kubectl.png

kubectl get all - apercu des ressources deployees

09

CI/CD - GitLab

Le depot est heberge sur GitLab et un pipeline CI/CD se declenche a chaque push. Trois stages s'enchainent : test, build, deploy.



Pipeline GitLab CI - 3 stages (test -> build -> deploy)

Stage test

Lance les 11 tests pytest du backend. Une **Postgres en service** est demarree par GitLab pour que les tests aient une vraie BDD a interroger. Le rapport JUnit est genere et expose dans l'interface GitLab (onglet Tests).

```
test-backend:
stage: test
image: python:3.11-slim
services:
- name: postgres:16-alpine
alias: postgres
script:
- cd backend
- pip install -r requirements.txt
- pytest -v --junitxml=report.xml
artifacts:
reports: { junit: backend/report.xml }
```

Stage build

Build des images Docker du backend et du frontend en parallele (needs: test-backend) avec Docker-in-Docker. Les images sont poussees vers la **GitLab Container Registry** avec deux tags : le SHA court du commit et latest.

Stage deploy

Pour ce projet etudiant, le deploiement reel sur un cluster est simule (on affiche les images publiees). Sur un projet pro on ferait kubectl apply ou helm upgrade. Cette etape ne se declenche que sur la branche main via une regle.

image manquante : screen_pipeline.png

Vue d'un pipeline reussi dans GitLab

10

Tests

Le backend est couvert par **11 tests pytest** qui exercent les principales routes. Les tests utilisent **TestClient** de FastAPI comme un context manager pour declencher le lifespan (init BDD, chargement initial). En CI, une vraie Postgres est utilisee comme service.

Tests existants

- **test_hello** : healthcheck
- **test_movies_returns_list** / **_default** / **_have_id** : route /movies
- **test_movie_detail** : detail d'Inception (id 27205)
- **test_movie_not_found** : id inconnu -> {error}
- **test_search** / **_no_results** : recherche par mot-cle
- **test_export_json** / **_csv** : exports avec headers Content-Disposition
- **test_status** : route /status (uptime, mode, films_en_db)

Exemple

```
def test_hello(client):
resp = client.get("/hello")
assert resp.status_code == 200
data = resp.json()
assert data["status"] == "ok"
assert data["mode"] in ("tmdb", "mock")
assert data["films_en_db"] > 0
```

11

Difficultes rencontrees et solutions

Cette section regroupe les problemes concrets que nous avons rencontres et la maniere dont nous les avons resolus. C'est la partie la plus instructive de ce projet.

Le backend demarrait avant Postgres

Avec Docker Compose, le backend se lancait avant que Postgres ait fini d'initialiser sa BDD. Resultat : crash au demarrage avec connection refused.

> **Solution :** ajout d'un healthcheck Postgres dans le compose (pg_isready) et utilisation de depends_on: condition: service_healthy. En complement, le backend possede une boucle wait_for_db() qui retry 30 fois pour les cas ou Compose ne suffit pas (cas Kubernetes notamment).

Migration SQLite -> PostgreSQL

Au depart on avait commence avec SQLite parce que c'est plus simple (juste un fichier). Mais le sujet du projet demande explicitement PostgreSQL, MySQL, MongoDB, etc. Il a fallu migrer.

> **Solution :** remplacement de sqlite3 par psycopg2. Le plus delicat etait les placeholders (? -> %s) et le fait que psycopg2 utilise un cursor explicite. Heureusement la clause ON CONFLICT DO UPDATE existe dans les deux moteurs, le code SQL n'a quasi pas change.

Les tests echouaient en CI sans BDD

Une fois passe a Postgres, les tests pytest ne pouvaient plus tourner sans un serveur de BDD lance.

> **Solution :** ajouter un service: postgres:16-alpine dans le job test du .gitlab-ci.yml. GitLab demarre alors le conteneur Postgres en parallele du conteneur Python et expose l'alias postgres comme nom de hote.

Erreur YAML dans le pipeline

Au debut, le job build-backend echouait avec une erreur de syntaxe YAML parce qu'on avait ecrit un docker tag sur plusieurs lignes avec des &&.

> **Solution :** remettre la commande docker tag SOURCE DEST sur une seule ligne, ce que YAML attendait.

PageBreak() oublie dans le rapport PDF

Lors de la generation du PDF, le contenu du sommaire se superposait avec la page de garde. La fonction onFirstPage dessinait la page de garde mais le contenu du story commencait aussi sur la meme page.

> **Solution :** ajouter un PageBreak() en tout premier element du story, ce qui force le contenu a commencer page 2 tandis que onFirstPage peint sa page de garde page 1.

Choix entre framework et vanilla JS

On a hesite a utiliser React pour le front. On a finalement garde vanilla JS parce que :

- L'application n'a pas besoin de gestion d'etat complexe
- On voulait eviter d'ajouter un build step (npm install, vite build)
- On preferait travailler directement sur fetch() / DOM pour mieux comprendre

Si on devait recommencer sur un projet plus gros, on partirait sur Vite + React.

Migration GitHub -> GitLab

Le projet a debute sur GitHub puis a ete migre sur GitLab. La pipeline github actions a ete reecrite en .gitlab-ci.yml, et le push initial necessitait un Personal Access Token avec scope write_repository.

12

Bilan et perspectives

Ce qui fonctionne bien

- Lancement complet du projet en une seule commande (./start.sh)
- 11 tests automatises qui passent en CI sur une vraie Postgres
- Deploiement K8s reproductible via ./k8s/deploy.sh
- Mode mock automatique : l'application demarre meme sans cle TMDB
- Persistance des donnees (volume Compose / PVC K8s) - les films restent
- Cache des details TMDB en BDD : on n'appelle l'API qu'une seule fois par film

Ce qu'on aurait fait avec plus de temps

- Mode favoris persistes (table users_favorites)
- Recommandations "films similaires" (basees sur le genre / realisateur)
- Cache Redis pour les requetes **/movies** frequentes
- Frontend reecrit en React + Vite pour gagner en organisation
- Authentification (JWT) pour proteger /export et l'admin
- TLS sur l'Ingress avec cert-manager + Let's Encrypt en prod

Ce que le projet nous a appris

- Structurer une application en 3 couches et choisir des outils coherents
- Manipuler une vraie BDD relationnelle (schema, requetes parametrees, upsert)
- Lire et appliquer une documentation d'API externe (TMDB)
- Conteneuriser une application multi-services et gerer les dependances
- Comprendre les concepts cles de Kubernetes (Pods, Services, Ingress, HPA)
- Mettre en place une CI/CD complete avec services et registry
- Travailler en binome avec un depot git partage

A

Annexes - commandes utiles

Lancement local

```
# tout lancer
./start.sh # ou : docker compose up --build

# en arriere-plan
docker compose up -d

# logs
docker compose logs -f backend

# tout arreter (et supprimer le volume postgres)
docker compose down -v
```

Tests

```
# postgres doit tourner pour les tests
docker compose up -d db
cd backend
pip install -r requirements.txt
pytest test_main.py -v
```

Deploiement Kubernetes

```
cd k8s
./deploy.sh # installe tout, attend que les pods soient prets

# observer
kubectl -n cinescope get all
kubectl -n cinescope get hpa
kubectl -n cinescope get ingress
kubectl -n cinescope logs -l tier=backend --tail=50

# scaler manuellement
kubectl -n cinescope scale deployment backend --replicas=4

# acceder via Ingress
echo "$(minikube ip) cinescope.local" | sudo tee -a /etc/hosts
# puis ouvrir http://cinescope.local

# nettoyer
./teardown.sh
```

Quelques requetes utiles

```
curl http://localhost:8000/hello
curl http://localhost:8000/status
curl 'http://localhost:8000/movies?limit=5'
curl 'http://localhost:8000/search?q=nolan'
curl http://localhost:8000/export/movies.csv -o films.csv
```

Structure du depot

```
.
+-- backend/ API FastAPI
| +-- main.py
| +-- test_main.py
| +-- requirements.txt
| +-- Dockerfile
+-- frontend/ SPA legere (HTML/CSS/JS)
+-- k8s/ Manifests Kubernetes
| +-- namespace.yaml
| +-- configmap.yaml secret.yaml
| +-- postgres-{deployment,service,pvc}.yaml
| +-- backend-{deployment,service,hpa}.yaml
| +-- frontend-{deployment,service,hpa}.yaml
| +-- ingress.yaml
| +-- deploy.sh teardown.sh
+-- docs/ Generation du rapport PDF
+-- docker-compose.yml
+-- .gitlab-ci.yml
+-- start.sh
+-- README.md
```

Rapport genere automatiquement avec reportlab + matplotlib.
Polices : Inter et Poppins (memes que le frontend).